EECS 4314 E - Advanced Software Engineering
Project Final Report - Group 10

Group Members:
- Manjot Kaur
- Gurnoor Kahlon
- Nathan Kwok
- Andriy Lytvyn
- Larissa Singh
- Rishab Yadav

Demo Video Link: https://www.youtube.com/watch?v=4wgShlmctOQ

GitHub Repository: https://github.com/Digie3/Savr

Instructions for Accessing the Project (on GitHub as well):

Step 1) Install the prerequisite software:

A. Install WSL 2 (Windows-only):
- Open PowerShell as Administrator and run: **text wsl --install**
- Restart your computer after installation (if needed).
- Verify WSL 2 (ignore WSL 1 message): **text wsl --status**

B. Install Docker Desktop:
- Download and install Docker Desktop.
- Open Docker Desktop & accept terms and conditions.
- Restart your computer after installation (if needed).
- Verify Docker Installation: **text docker --version**
  - Note: The other softwares needed are installed using Docker build (more info below).

C. Install Git.

Step 2) Clone the Repository.

Step 3) Environment Configuration:

- The backend reads its settings from a .env file. Copy the provided template:
  a. cd .../backend
  b. cp .env.example .env

Step 4) Run Docker:

a. Open Docker Desktop.
b. Build & Run (if you haven't built already): text docker compose up --build
- Note: Building Docker is mandatory, it installs all the software needed (outside of the prerequisite software installation).

c. Run (if you have built already): text docker compose up
d. Close Docker Containers: text docker compose down

1. Introduction and Project Objective:

Savr is a recipe-sharing social media platform designed to make recipe discovery, recipe publishing, and food-related interaction easier in one application. The project combines the familiar structure of a social media feed with recipe-specific features such as ingredients, preparation steps, recipe images, saved recipes, comments, ratings, following, and creator statistics. The main objective is to give users a central place where they can create and share recipes, browse recipes from other users, save recipes for later, interact with recipes, and follow creators whose content they like.

The motivation for the project is that recipes are often scattered across websites, short videos, screenshots, and personal notes. Savr organizes recipe content into a structured application where each recipe has a title, description, ingredients, instructions, images, social interactions, and creator information. This makes the content easier to browse, compare, revisit, and analyze. The project also includes an analytics and lakehouse component so user activity and recipe engagement can be tracked and processed separately from the main transactional database.

2. Scope of the Final Deliverable:

Our final deliverable includes a working codebase, a demonstration video, design artifacts, traceability tables, and testing evidence. The implemented application includes user authentication, recipe creation, recipe feed and detail pages, recipe deletion, saved recipes, following, comments, ratings, sorting/statistics, image search, profile-related features, analytics, and a local data lakehouse pipeline. The repository also includes Docker support so the project can be built and run more consistently across machines. The project uses SQLite as the operational database for transactional application data. This includes users, recipes, ingredients, comments, ratings, followers, saved recipes, and media references. The analytics/lakehouse layer is separate from the operational database and follows a medallion-style structure with bronze, silver, and gold stages. This separation allows the core application to remain focused on user-facing transactions while analytics data can be cleaned, transformed, and summarized for reporting and future insights.

3. Technology Stack:

Area	**Technology Used**	**Purpose**
Frontend	React with Vite	Builds the single page application, navigation, recipe feed, forms, saved recipes, profile screens,

		dashboards, and detail pages.
Backend	Node.js with Express	Provides REST APIs for authentication, recipes, saved recipes, follows, comments, ratings, analytics, creator statistics, and image search.
Operational Database	SQLite	Stores transactional application data used directly by the application.
Authentication	JWT, bcrypt, auth middleware	Protects private features and stores passwords as hashes instead of plaintext.
Uploads	Local backend upload folders	Stores recipe, ingredient, instruction-step, and profile images for the project demo.
Image Service	Google Search API support with local fallback service	Provides ingredient image retrieval and keeps the app usable when external API keys are unavailable.
Analytics / Lakehouse	ActivityEvents, Python, PySpark, Apache Spark, Delta Lake-style medallion pipeline	Captures and processes activity data into bronze, silver, and gold analytics outputs.
Testing	Node test runner	Provides automated test cases for major backend features and service behavior.
Deployment / Setup	Docker and docker compose	Provides repeatable setup and local demonstration steps.

4. Functional Requirements RTM:

The following RTM links functional requirements to implementation evidence, demonstration evidence, and testing evidence. This table is included to show that the implemented features are traceable from requirement to code and test coverage.

ID	Requirement	Description	Implementation Evidence	Demo/API Evidence	Test Evidence	Status
FR1	User Registration	New users can create an account and passwords are not stored in plaintext.	auth routes/controllers /services, Users table	Register page, POST /register	auth.test.mjs	Met
FR2	User Login	Existing users can log in and receive an authenticated session/token .	auth routes/controllers /services	Login page, POST /login	auth.test.mjs	Met
FR3	Protected User Session	Private features require authenticatio n.	requireAuth middleware, ProtectedRoute.j sx	Profile/Creat e/Saved/Anal ytics/Creator/ Following	auth.test.mjs	Met
FR4	Profile View/Edit	Users can view and update profile information.	user routes/controllers /services, Profile.jsx	Profile page and user endpoints	Manual/dem o evidence	Met
FR5	Profile Image Upload	Users can upload/retriev e profile images.	profile image upload middleware, user controller	profile image endpoints	Manual/dem o evidence	Met
FR6	Create Recipe	Authenticated users can create recipe posts.	CreateRecipe.jsx , recipe routes/controllers /services	Create page, POST /create	createRecipe .test.mjs	Met
FR7	Recipe Ingredients	Recipes store ingredient details such as quantity, unit, and description.	Ingredients and Recipes_has_Ing redients tables	Create form and recipe detail	createRecipe .test.mjs	Met

FR8	Recipe Steps	Recipes store ordered preparation instructions.	RecipeSteps table, RecipeDetails.jsx	Create form and recipe detail	createRecipe.test.mjs	Met
FR9	Recipe Image Uploads	Recipes support main, ingredient, and instruction-step images.	Media and recipe media tables, upload middleware	Create recipe image controls	createRecipe and image tests	Met
FR10	Recipe Feed	Users can browse recipes from all users in a home feed.	Home.jsx, RecipeCard.jsx, recipe service	Home page, GET /recipes	sortingStats.test.mjs	Met
FR11	Recipe Detail View	Users can open a recipe and view full content.	RecipeDetails.jsx, recipe controller/service	GET /recipes/:id	commentRating.test.mjs indirectly	Met
FR12	Delete Recipe	Recipe owners can delete their own recipes.	recipe delete controller/service logic	DELETE /recipes/:id	deleteRecipe.test.mjs	Met
FR13	Delete Authorization	Users cannot delete recipes they do not own.	ownership validation and auth middleware	DELETE /recipes/:id	deleteRecipe.test.mjs	Met
FR14	Save Recipe	Authenticated users can save recipes.	saved recipe routes/controllers/services	POST /recipes/:id/save	savedRecipe.test.mjs	Met
FR15	View Saved Recipes	Users can view saved recipes later.	SavedRecipes.jsx and saved recipe service	GET /saved-recipes	savedRecipe.test.mjs	Met
FR16	Unsave Recipe	Users can remove recipes from saved list.	saved recipe routes/controllers/services	DELETE /recipes/:id/save	savedRecipe.test.mjs	Met

FR17	Follow User	Users can follow other creators.	follow routes/controllers /services	POST /follow/:idFollowed	follow.test.mjs	Met
FR18	Unfollow User	Users can unfollow creators.	follow service	DELETE /follow/:idFollowed	follow.test.mjs	Met
FR19	Follower/Following Lists	Users can view follower/following relationships.	Followers table and follow endpoints	followers/following endpoints	follow.test.mjs	Met
FR20	Following Feed	Users can view recipes from followed users.	FollowingFeed.jsx, follow service	GET /following-feed	follow.test.mjs	Met
FR21	Create Comment	Users can comment on recipes.	comments routes/controllers /services	POST /recipes/:id/comments	commentRating.test.mjs	Met
FR22	Edit/Delete Comment	Comment owners can edit or delete their own comments.	comment service ownership checks	comment edit/delete endpoints	commentRating.test.mjs	Met
FR23	Comment Authorization	Non-owners cannot edit/delete another user comment.	comment ownership validation	comment edit/delete endpoints	commentRating.test.mjs	Met
FR24	Submit Rating	Users can rate recipes from 1 to 5.	rating routes/controllers /services	POST /recipes/:id/rating	commentRating.test.mjs	Met
FR25	Update Rating	A user changing their rating updates the existing rating.	rating service update/upsert behavior	rating endpoint	commentRating.test.mjs	Met

FR26	Average Rating	Recipes return aggregate rating data.	recipe/rating services	feed and recipe detail responses	commentRating/sortingStats tests	Met
FR27	Sorting by Date	Recipe feed supports newest/oldest sorting.	recipe service sorting query	GET /recipes sort params	sortingStats.test.mjs	Met
FR28	Sorting by Rating	Recipe feed supports highest-rated sorting.	recipe service aggregate query	GET /recipes?sort=rating	sortingStats.test.mjs	Met
FR29	Sorting by Views	Recipe feed supports most-viewed sorting.	ActivityEvents and recipe service	GET /recipes?sort=views	sortingStats.test.mjs	Met
FR30	Creator Statistics	Creators can view dashboard/statistics data.	creator routes/controllers/services, CreatorDashboard.jsx	creator dashboard endpoints	Manual/demo evidence	Met
FR31	Analytics Dashboard	The app displays analytics and event summaries.	Analytics.jsx, analytics controller/service	analytics dashboard endpoint	Analytics implementation	Met
FR32	Activity Logging	The system captures activity events for analytics.	ActivityEvents table, activity logging helpers	POST /activity/internal logging	Analytics/lakehouse implementation	Met
FR33	Ingredient Image Search	Users can search for ingredient images.	image routes/controllers/services, IngredientImageSearch.jsx	GET /images/search	imageSearch/imageService tests	Met
FR34	Image Fallback	Image feature works without	fallbackImageService and	fallback image endpoint	fallbackImages/imageService tests	Met

		external API keys.	fallback SVG route			
FR35	Persist Image URL	Selected image URLs can be stored with recipe media.	Media table and recipe service	Create recipe submission	image tests	Met
FR36	Lakehouse Bronze	Raw activity data can be captured in a bronze layer.	lakehouse bronze pipeline	lakehouse ETL	Documentation/manual run	Met
FR37	Lakehouse Silver	Raw data can be cleaned into structured data.	lakehouse silver pipeline	lakehouse ETL	Documentation/manual run	Met
FR38	Lakehouse Gold	Analytics outputs can be summarized for business questions.	lakehouse gold pipeline	lakehouse ETL	Documentation/manual run	Met
FR39	Dockerized Setup	Project can be run using Docker.	docker compose files and README	docker compose up --build	Manual setup evidence	Met

5. Non-Functional Requirements RTM:

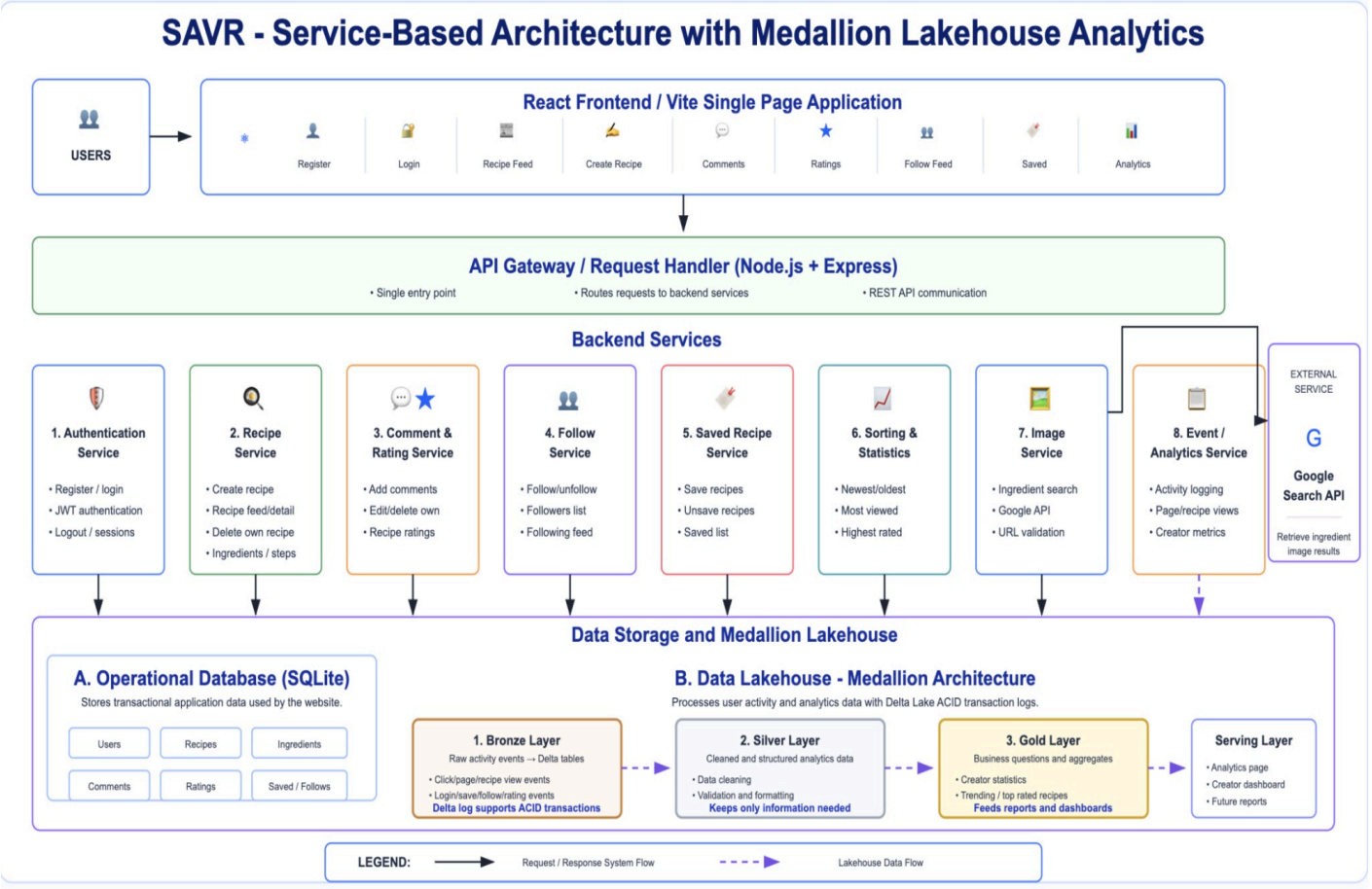
The non-functional requirements focus on maintainability, security, usability, deployment, testability, and analytics extensibility. These qualities are important because the project is a team-built application with multiple services and several user-facing workflows.

ID	**Requirement**	**Quality Goal**	**Implementation Evidence**	**Status**
NFR1	Maintainability	Code should be organized so future changes are easier.	Backend is split into routes, controllers, services, middleware, helpers, config, and tests.	Met

NFR2	Modularity	Features should have clear responsibility boundaries.	Separate modules exist for auth, recipes, saved recipes, follows, comments, ratings, analytics, creator stats, and image search.	Met
NFR3	Password Security	Passwords should not be plaintext.	Authentication uses hashing and tests verify plaintext is not stored.	Met
NFR4	Protected Access	Private routes should require authentication.	requireAuth middleware and ProtectedRoute protect private pages and endpoints.	Met
NFR5	Authorization	Users should not modify resources they do not own.	Delete recipe and comment edit/delete validate ownership.	Met
NFR6	Input Validation	Invalid requests should return clear errors.	Auth, image search, comment, rating, and recipe endpoints validate input.	Met
NFR7	Data Consistency	Related data should stay consistent.	SQLite schema separates users, recipes, ingredients, media, comments, ratings, follows, and saves.	Met
NFR8	Testability	Core behavior should have automated tests.	Backend tests cover major features and service behavior.	Met
NFR9	Deployability	Project should run consistently on different machines.	Docker and docker compose setup were added.	Met

NFR10	Usability	Main workflows should be understandable.	Frontend navigation and pages support recipe browsing, creation, saved recipes, profile, creator, and analytics.	Met
NFR11	UI Consistency	Pages should share a consistent structure.	Shared navigation, recipe cards, forms, and dashboard patterns are used.	Met
NFR12	Analytics Extensibility	Future activity types should be trackable.	ActivityEvents stores event type, actor, target, timestamp, and metadata.	Met
NFR13	External API Resilience	The app should work without paid/API-key access..	Image search has fallback images when Google configuration is unavailable.	Met
NFR14	Observability	Developers should quickly verify backend health.	Backend includes a health/status route.	Met

6. Architecture and Design Explanation:



Savr uses a layered, service-based architecture. The React frontend handles the user interface and calls the backend through REST APIs. The Express backend is organized into routes, controllers, services, middleware, helpers, and configuration files. Routes define API endpoints, controllers manage request/response flow, and services contain the main business logic. This structure keeps the code easier to understand and supports team collaboration because each feature can be developed in its own area.

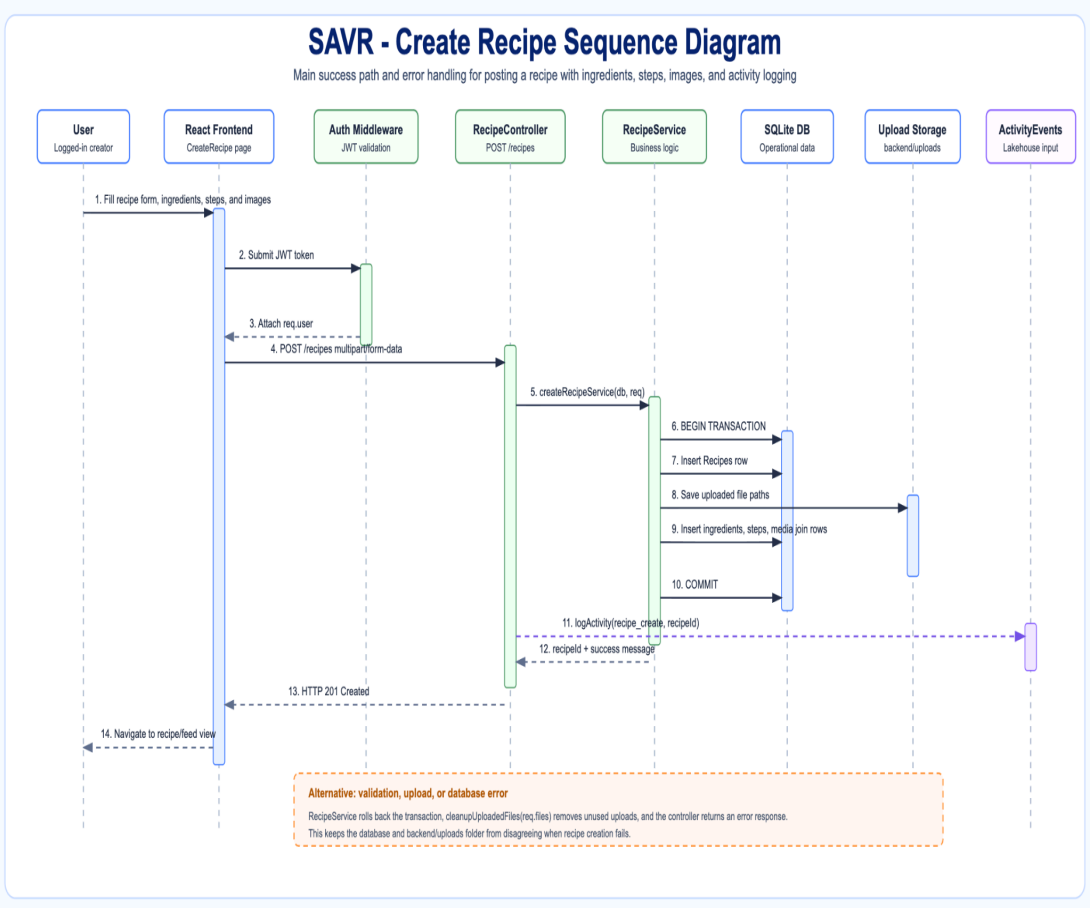
The operational database is SQLite. It stores the application data required for the website to work, including users, recipes, ingredients, recipe steps, media, comments, ratings, followers, and saved recipes. Uploaded images are stored in backend upload folders and referenced through database media records. This design is suitable for a course project because it is lightweight, easy to run locally, and clear to demonstrate.

The analytics side uses an event collection and lakehouse approach. The software used are Apache Spark and PySpark for the processing and storing of data, while Delta Lake provides the ACID transactions needed for a lakehouse shown in the delta log folder. User activity events can be stored and later processed through a medallion pipeline. The bronze layer keeps raw activity data, the silver layer cleans and structures that data, and the gold layer summarizes it into analytics outputs such as usage metrics or creator/statistics questions. This

mirrors a real analytics architecture while keeping the main application database focused on transactional operations.

Layer	Components	Responsibility
Frontend Layer	React/Vite pages and components	Displays application screens and sends API requests.
API Layer	Express route files	Defines endpoints for each feature area.
Controller Layer	Backend controllers	Handles request parsing, responses, and calls services.
Service Layer	Backend service modules	Contains business rules, validation, ownership checks, sorting, statistics, and database work.
Middleware/Helpers	Auth middleware, upload middleware, helpers	Protects routes, manages uploads, and supports shared backend logic.
Operational Database	SQLite relational tables	Stores app transactions and user-facing data.
File Storage	Backend uploads folders	Stores uploaded recipe/profile images for demo use.
Analytics/Lakehouse	ActivityEvents, bronze, silver, gold pipeline	Captures and processes usage data separately from operational data.
External Service	Google Search API with fallback image service	Retrieves ingredient images and provides fallback behavior.

7. Sequence Diagram Explanation:

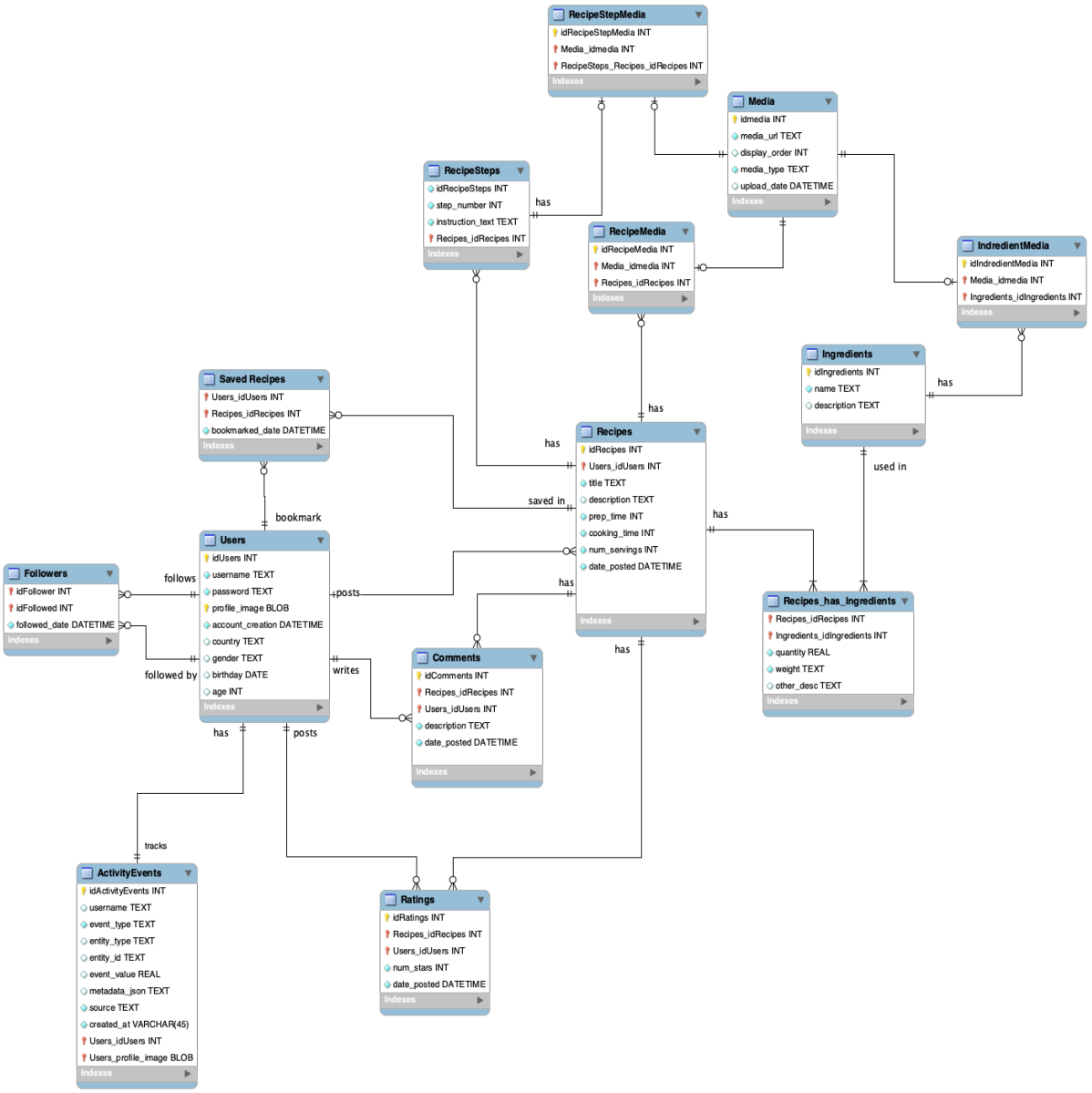


The sequence diagrams should focus on specific workflows rather than the entire system at once. Recommended workflows include Create Recipe, Save Recipe, Delete Recipe, Comment/Rating, Image Search, and Analytics/Lakehouse Processing. These flows are different from the architecture diagram because they show the order of communication between the user, frontend, backend, services, database, and external or analytics layers.

Workflow	**Main Sequence**	**Reason for Inclusion**
Create Recipe	User submits recipe form -> React sends request -> recipe route/controller receives request -> recipe service validates/stores recipe, ingredients, steps, and media -> SQLite stores	This is one of the core application workflows.

	records -> response returns to frontend.	
Delete Recipe	Owner clicks delete -> frontend sends authenticated delete request -> backend checks user identity and ownership -> service deletes recipe/dependent records -> frontend refreshes feed/detail page.	Shows authentication, authorization, and destructive action handling.
Save Recipe	User clicks save -> frontend sends request -> saved recipe service creates relation between user and recipe -> saved page later retrieves saved recipes.	Shows user-specific recipe interaction.
Comment and Rating	User submits comment/rating -> backend validates input and auth -> service stores comment/rating -> recipe details update with new interaction data.	Shows social interaction around recipe posts.
Image Search	User searches ingredient image -> frontend calls image endpoint -> backend uses Google Search API or fallback service -> image choices return to frontend -> selected image URL can be stored.	Shows external service integration and fallback design.
Analytics/Lakehouse	Frontend/backend event occurs -> event is logged -> raw data enters bronze -> cleaned in silver -> summarized in gold -> dashboard/statistics can use outputs.	Shows how analytics is separated from operational transactions.

8. Class Diagram Explanation:



The class diagram for Savr represents the main domain entities of the application and their relationships. Since Savr is implemented using JavaScript modules, Express services, and a SQLite relational database rather than strict object-oriented classes, the diagram is best understood as a class-style domain model or entity relationship model.

The diagram includes the main application entities such as Users, Recipes, Ingredients, RecipeSteps, Media, Comments, Ratings, Saved Recipes, Followers, and ActivityEvents. These entities show how the core data in the system is connected. For example, a User can create many Recipes, write Comments, submit Ratings, save Recipes, follow other Users, and generate ActivityEvents. A Recipe belongs to a User and can have Ingredients, RecipeSteps,

Media, Comments, Ratings, and SavedRecipe records. Also, the different types of media entities are inheriting the parent Media's fields.

Although the backend services are not all drawn directly as classes in the diagram, they interact with these entities in the implementation. For example, AuthService works with Users, RecipeService works with Recipes, Ingredients, RecipeSteps, and Media, CommentRatingService works with Comments and Ratings, FollowService works with Followers, SavedRecipeService works with Saved Recipes, and AnalyticsService works with ActivityEvents and lakehouse data.

This diagram is still useful for the design document because it shows the structure of the application data and the relationships between the major objects in the system. Since the project is not written as a traditional OOP application, the diagram focuses more on domain entities and database relationships, while the service layer is explained separately as the logic that creates, updates, retrieves, validates, and deletes those entities.

9. Demonstration Coverage:

Our video shows both the user-facing features and the supporting engineering work. It begins with the project motivation, then shows login/register, recipe creation, the home feed, recipe details, saving, following, commenting, rating, deleting a recipe, sorting/statistics, image search, analytics/lakehouse explanation, test cases, and repository structure. Each feature is tied back to at least one requirement in the RTM.

Rubric Area	**What to Demonstrate**	**Project Evidence**
Objective/Motivation	Explain why Savr exists and what problem it solves.	Recipe-sharing social platform with structured recipe content.
Requirements	Show major functional requirements working.	RTM and live workflows.
Architecture	Explain frontend, backend services, database, uploads, image service, and lakehouse.	Architecture/design section and repo structure.
Design Artifacts	Show architecture, sequence, and class/domain diagrams.	Report and diagram slides/files.
Testing	Show test files and explain outcomes.	Backend tests folder and test table.

Repository Practices	Show GitHub repo, README, Docker instructions, tests, and organized folders.	GitHub repository.
Retrospective	Explain what was learned and future improvements.	Future work section.

10. Detailed Test Case Table:

The test cases below are tied to the RTM requirements. They include automated backend tests where available and manual/demo verification where the feature is primarily UI-driven or setup-driven.

Test ID	**Requirement(s)**	**Test Case**	**Input / Setup**	**Expected Outcome**	**Evidence**
TC1	FR1	Register valid user	Unique username/email and valid password	201 response; user created	auth.test.mjs
TC2	FR1	Password hashing	Register user and inspect stored password	Stored password differs from plaintext	auth.test.mjs
TC3	FR1	Duplicate registration	Register same account twice	Second request rejected	auth.test.mjs
TC4	FR2	Valid login	Use registered credentials	Login succeeds with token/session	auth.test.mjs
TC5	FR2	Invalid login	Use wrong password	Login rejected	auth.test.mjs
TC6	FR3	Protected route without auth	Call protected endpoint without token	Request rejected	auth.test.mjs

TC7	FR3	Protected route with auth	Call protected endpoint with token	Request succeeds	auth.test.mjs
TC8	FR6	Create recipe successfully	Authenticated user submits recipe with ingredients/steps	Recipe created	createRecipe.test.mjs
TC9	FR7/FR8	Store ingredients and steps	Create recipe with ingredient and instruction data	Related records stored and returned	createRecipe.test.mjs
TC10	FR9	Store recipe media	Create recipe with image data	Media references stored	createRecipe.test.mjs/image tests
TC11	FR12	Owner deletes recipe	Owner sends delete request	Recipe removed	deleteRecipe.test.mjs
TC12	FR13	Non-owner delete blocked	Different user sends delete request	Authorization error	deleteRecipe.test.mjs
TC13	FR14	Save recipe	Authenticated user saves recipe	Saved recipe relation created	savedRecipe.test.mjs
TC14	FR15	View saved recipes	Open saved recipes list	Saved recipe appears	savedRecipe.test.mjs
TC15	FR16	Unsave recipe	Remove saved recipe	Saved recipe removed	savedRecipe.test.mjs
TC16	FR17	Follow user	User follows creator	Follower relation created	follow.test.mjs

TC17	FR18	Unfollow user	User unfollows creator	Follower relation removed	follow.test.mjs
TC18	FR19	Follower/following lists	Request follower/following endpoints	Correct users returned	follow.test.mjs
TC19	FR20	Following feed	Followed user has recipes	Feed returns followed-user recipes	follow.test.mjs
TC20	FR21	Create comment	Authenticated user submits valid comment	Comment stored	commentRating.test.mjs
TC21	FR21	Reject unauthenticated comment	Submit comment without token	Request rejected	commentRating.test.mjs
TC22	FR21	Reject empty/invalid comment	Submit blank/too long comment	Validation error	commentRating.test.mjs
TC23	FR22	Edit own comment	Owner updates comment text	Comment updated	commentRating.test.mjs
TC24	FR22	Delete own comment	Owner deletes comment	Comment removed	commentRating.test.mjs
TC25	FR23	Block non-owner comment edit/delete	Different user attempts edit/delete	Authorization error	commentRating.test.mjs
TC26	FR24	Submit valid rating	Authenticated user rates 1-5	Rating stored and aggregate returned	commentRating.test.mjs

TC27	FR24	Reject invalid rating	Submit rating outside 1-5	Validation error	commentRating.test.mjs
TC28	FR25	Update existing rating	Same user rates same recipe again	Rating updated, not duplicated	commentRating.test.mjs
TC29	FR26	Average rating returned	Fetch recipe after ratings	Average/count included	commentRating/sortingStats tests
TC30	FR27	Sort newest/oldest	Seed recipes with dates and request sort	Correct date order returned	sortingStats.test.mjs
TC31	FR28	Sort by rating	Seed recipes with different ratings	Highest-rated recipes first	sortingStats.test.mjs
TC32	FR29	Sort by views	Seed view activity events	Most-viewed recipes first	sortingStats.test.mjs
TC33	FR33	Image search requires auth	Call image search without auth	Request rejected	imageSearch.test.mjs
TC34	FR33	Image search validates query	Call with blank/missing/long query	Validation error	imageSearch.test.mjs
TC35	FR34	Fallback images without API keys	Run image service without Google config	Fallback results returned	fallbackImages/imageService tests
TC36	FR34	Fallback SVG renders safely	Request fallback image with unsafe text	Valid escaped SVG returned	fallbackImages.test.mjs
TC37	FR33	Google result mapping	Mock Google response	Mapped image results returned	imageService.test.mjs

TC38	FR33	Image cache behavior	Repeat same image request	Cached result used where applicable	imageService.test.mjs
TC39	FR33	External image failure handling	Mock Google/API/network failure	Controlled error response, no crash	imageService.test.mjs
TC40	FR35	Persist selected image URL	Submit safe external/fallback URL in recipe	URL stored with media	image tests
TC41	FR31/FR32	Analytics event logging	Trigger activity event	Event stored/summarized	Analytics implementation/manual demo
TC42	FR36-FR38	Lakehouse pipeline run	Run ETL pipeline manually or through Docker	Bronze/silver/gold outputs generated	Manual/Docker verification
TC43	FR39	Docker build/run	Run docker compose up --build	Frontend/backend start successfully	Manual setup verification

11. Test Outcome Summary:

The repository includes automated backend tests for authentication, create recipe, delete recipe, saved recipes, follows, comments/ratings, sorting/statistics, image search, fallback images, and image service behavior. Focused image service tests were verified locally in this environment and passed. Some server-based integration tests may depend on local server startup and health-check timing; if the final team runs the project through Docker, the final submission should include the successful local or Docker test output as additional evidence.

Test Area	**Files**	**Coverage**	**Status / Note**
Authentication	auth.test.mjs	Registration, login, password hashing, protected current user behavior	Automated tests included.

Create Recipe	createRecipe.test.mjs	Recipe creation, ingredients, steps, and media	Automated tests included.
Delete Recipe	deleteRecipe.test.mjs	Owner delete and non-owner authorization rejection	Automated tests included.
Saved Recipes	savedRecipe.test.mjs	Save, view saved, unsave	Automated tests included.
Follow Service	follow.test.mjs	Follow, unfollow, follower/following lists, following feed	Automated tests included.
Comments/Ratings	commentRating.test.mjs	Comment validation, edit/delete authorization, rating validation/aggregates	Automated tests included.
Sorting/Statistics	sortingStats.test.mjs	Sorting by date, rating, and views	Automated tests included.
Image Service	imageSearch.test.mjs, imageService.test.mjs, fallbackImages.test.mjs	Validation, Google result mapping, fallback images, caching, error handling	Focused image/fallback service tests passed locally.
Lakehouse	lakehouse ETL scripts	Bronze, silver, and gold pipeline behavior	Manual/Docker verification recommended for final evidence.

Demonstrated Test Case Outcomes

The project includes automated backend test cases tied to the functional requirements listed in the RTM. The screenshots below show example test execution outcomes from the project test suite. These outcomes demonstrate that key backend features were tested, including authentication, recipe creation, recipe deletion, saved recipes, follows, comments/ratings, sorting/statistics, and image service behavior. The detailed test case table above explains the full set of test cases, the related requirement IDs, the input/setup used, and the expected outcome for each test. The screenshots below provide demonstrated evidence of test execution, while the detailed test case table provides the complete traceability between test cases and RTM requirements.

Example Test Case 1: Backend test execution output using npm test:

```
> savr-backend@0.1.0 test
> node --test

✔ registers a new user and stores the password hashed, not in plaintext (811.442416ms)
✔ rejects a duplicate username with 409 (196.154125ms)
✔ rejects missing fields and weak passwords with 400 (6.073583ms)
✔ login rejects bad credentials and returns a token for valid ones (216.2845ms)
✔ protected /me and /logout require a valid token (168.795625ms)
✔ rejects unauthenticated comment creation (738.694416ms)
✔ rejects an empty / whitespace-only comment (3.383ms)
✔ rejects an overly long comment (2.037667ms)
✔ creates a valid comment (3.348375ms)
✔ commenting on a nonexistent recipe returns 404 (2.356625ms)
✔ the comment owner can edit their comment (4.1275ms)
✔ a different user cannot edit someone else's comment (2.757333ms)
✔ editing a nonexistent comment returns 404 (1.730625ms)
✔ a different user cannot delete someone else's comment (2.236709ms)
✔ the comment owner can delete their comment (2.520167ms)
✔ rejects an unauthenticated rating (0.89575ms)
✔ rejects ratings outside 1–5 (including decimals and strings) (5.084625ms)
✔ submits a valid rating and returns average + count (3.862584ms)
✔ re-rating updates the existing row instead of duplicating it (4.381959ms)
✔ recipe detail returns the viewer's own rating (3.12975ms)
✔ rating a nonexistent recipe returns 404 (1.185875ms)
✔ authenticated user can create a recipe with ingredients and steps (656.13275ms)
✔ recipe creation rejects a submission without ingredients (4.665042ms)
✔ recipe owner can delete their recipe (848.232583ms)
✔ different user cannot delete someone else's recipe (6.746541ms)
✔ exact match is returned first (0.567541ms)
✔ partial 'contains' query matches multiple ingredients (23.946833ms)
✔ startsWith query matches (0.187916ms)
✔ blank query matches nothing (0.654584ms)
✔ respects the limit (0.173708ms)
✔ getFallbackImages always returns at least one suggestion (0.4355ms)
✔ getFallbackImages returns relative paths (no host or port) (0.276333ms)


✔ renderFallbackSvg returns an SVG for a known slug (0.255792ms)
✔ renderFallbackSvg handles unknown slugs generically (0.62575ms)
✔ renderFallbackSvg escapes XML-unsafe characters in the label (0.64775ms)
✔ authenticated user can follow another user (654.6945ms)
✔ following feed contains recipes from the followed creator (3.359125ms)
✔ authenticated user can unfollow and the creator disappears from the feed (8.41325ms)
✔ rejects an unauthenticated search (776.372333ms)
✔ rejects a missing query (4.169417ms)
✔ rejects a blank / whitespace query (3.015584ms)
✔ rejects an overly long query (18.940167ms)
✔ rejects invalid limit values (22.438667ms)
✔ falls back to built-in suggestions when no API keys are set (1.524083ms)
✔ fallback returns several matches for a partial query (1.862958ms)
✔ fallback always returns at least one suggestion for an unknown query (2.337375ms)
✔ fallback image route serves an SVG (2.378417ms)
✔ fallback image route is public (no auth required) (1.177083ms)
✔ a selected fallback image URL persists on the recipe (22.643417ms)
✔ stores a valid external ingredient image URL on the recipe (55.953792ms)
✔ ignores an unsafe (non-http) ingredient image URL (11.210125ms)
Google Custom Search returned status 403
✔ isImageSearchConfigured reflects presence of both env vars (1.79975ms)
✔ maps Google items into the stable image shape (0.913291ms)
✔ respects the requested limit (0.402959ms)
✔ caches results for identical query + limit (0.36525ms)
✔ throws a 502-tagged error when Google returns a non-ok response (0.654791ms)
✔ throws a 502-tagged error when the network call fails (0.14075ms)
✔ isValidExternalImageUrl accepts http/https and rejects everything else (0.831875ms)
✔ isInternalFallbackImagePath accepts only well-formed fallback paths (0.311458ms)
✔ isAllowedIngredientImageUrl accepts external https and internal fallback paths (0.346792ms)
✔ authenticated user can save a recipe (586.965042ms)
✔ saved recipes list contains the saved recipe (2.657916ms)
✔ authenticated user can unsave a recipe and it disappears from saved recipes (7.305166ms)
✔ sorts by newest first by default and oldest first with order=asc (242.265334ms)
✔ sorts by highest average rating when sort=rating (2.886583ms)
✔ sorts by most viewed when sort=views (2.969875ms)
✔ aggregates view counts and average ratings correctly from seeded data (2.080333ms)
✔ falls back to newest ordering for an unrecognized sort value (1.269083ms)
ℹ tests 68


ℹ suites 0
ℹ pass 68
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 1854.57875
```

Example Test Case 2: Image service / image search test execution output:

```
[running] node ~/Users/nathanmwu/Documents/GitHub/SWE/codebase/backend
TAP version 13
# Subtest: rejects an unauthenticated search
ok 1 - rejects an unauthenticated search
  ---
  duration_ms: 416.087083
  ...
# Subtest: rejects a missing query
ok 2 - rejects a missing query
  ---
  duration_ms: 2.320792
  ...
# Subtest: rejects a blank / whitespace query
ok 3 - rejects a blank / whitespace query
  ---
  duration_ms: 2.202667
  ...
# Subtest: rejects an overly long query
ok 4 - rejects an overly long query
  ---
  duration_ms: 1.94025
  ...
# Subtest: rejects invalid limit values
ok 5 - rejects invalid limit values
  ---
  duration_ms: 5.844916
  ...
# Subtest: falls back to built-in suggestions when no API keys are set
ok 6 - falls back to built-in suggestions when no API keys are set
  ---
  ---
  duration_ms: 2.086375
  ...
# Subtest: fallback returns several matches for a partial query
ok 7 - fallback returns several matches for a partial query
  ---
  duration_ms: 1.67175
  ...
# Subtest: fallback always returns at least one suggestion for an unknown query
ok 8 - fallback always returns at least one suggestion for an unknown query
  ---
  duration_ms: 1.576417
  ...
# Subtest: fallback image route serves an SVG
ok 9 - fallback image route serves an SVG
  ---
  duration_ms: 1.843333
  ...
# Subtest: fallback image route is public (no auth required)
ok 10 - fallback image route is public (no auth required)
  ---
  duration_ms: 1.284125
  ...
# Subtest: a selected fallback image URL persists on the recipe
ok 11 - a selected fallback image URL persists on the recipe
  ---
  duration_ms: 12.259166
  ...
# Subtest: stores a valid external ingredient image URL on the recipe
# Subtest: stores a valid external ingredient image URL on the recipe
ok 12 - stores a valid external ingredient image URL on the recipe
  ---
  duration_ms: 7.783541
  ...
# Subtest: ignores an unsafe (non-http) ingredient image URL
ok 13 - ignores an unsafe (non-http) ingredient image URL
  ---
  duration_ms: 5.5645
  ...
1..13
# tests 13
# suites 0
# pass 13
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 803.034417

[Done] exited with code=0 in 1.001 seconds
```

12. Demonstration:

Evidence of Course Software Engineering Practices

The project demonstrates several software engineering practices discussed in the course. Savr uses a layered architecture where the React frontend, Express API routes, controllers, services, middleware, database, and lakehouse pipeline are separated into clear responsibilities. The backend follows a service-based design so features such as authentication, recipes, comments/ratings, follows, saved recipes, analytics, and image search can be developed and tested independently. The project also uses authentication middleware, ownership validation, an RTM for requirements traceability, automated backend tests, Docker-based setup, and GitHub version control. These practices improve maintainability, testability, deployment consistency, and team collaboration.

Usability and UI Consistency

The user interface is organized around clear navigation and consistent workflows. Users can move between Home, Create Recipe, Saved Recipes, Profile, Creator/Statistics, Analytics, Login, and Register pages through the application navigation. Recipe content is shown through consistent recipe cards and detail views, so users can browse posts, open a recipe, save it, comment, rate, or delete it when authorized. Forms follow a similar structure across authentication, recipe creation, and interaction features, which helps keep the application predictable and easier to use.

Source Code Repository and Good SENG Practices

The project uses GitHub as the shared source code repository. The team worked through feature areas and merged work into the main codebase. The repository includes frontend code, backend code, database-related files, lakehouse files, Docker setup, documentation, diagrams, and tests. This supports the rubric requirement for source code repository usage and good software engineering practices.

Practice	**Evidence**	**Benefit**
Version control	GitHub repository and commit history	Shows collaboration and project evolution.
Layered architecture	Frontend, backend, database, lakehouse separation	Improves clarity and maintainability.
Service separation	Routes/controllers/services/middle ware organization	Keeps business logic modular.

Testing	Backend tests for major features	Supports validation and regression checking.
Traceability	RTM maps requirements to implementation and tests	Shows requirements were not only listed but implemented.
Docker support	docker compose setup	Makes setup more repeatable for team and TA.
Documentation	README and report instructions	Helps evaluators run and understand the project.
Security basics	bcrypt, JWT, protected routes, ownership checks	Addresses common security needs for user accounts and user-owned content.
Fallback design	Image search fallback service	Improves reliability when external API keys are missing.

13. Retrospective and Future Work:

The project helped our team practice building a full-stack application with multiple coordinated services. Important learning areas included React/Vite frontend development, Express API design, SQLite schema design, authentication, file uploads, relational data modeling, automated testing, Docker setup, and separating analytics processing from transactional application logic. Our team also learned the importance of keeping feature ownership clear, testing after merges, and documenting setup steps early so the project can run consistently on different machines.

With more time, the project could be improved by deploying the full system online, replacing local image storage with cloud object storage, moving from SQLite to a production database such as PostgreSQL, expanding analytics dashboards, automating the lakehouse pipeline, adding stronger CI testing, and improving UI polish for forms and image controls. The recommendation features could also be expanded using the gold analytics outputs from the lakehouse layer.

Area	Current Limitation	Future Improvement
Hosting	Project may rely on local/Docker execution if not deployed.	Deploy frontend/backend/database and provide a stable public URL.
Database	SQLite is lightweight but not production-scale.	Move to PostgreSQL or another production database.
Image Storage	Images are stored in backend folders.	Use cloud object storage and signed URLs.
Image API	Google API keys and quotas may limit usage.	Add production key management and licensing checks.
Lakehouse	Pipeline may require manual or scheduled run.	Automate ETL jobs and dashboard refreshes.
Testing	Some integration tests may depend on startup timing.	Stabilize server lifecycle and add CI pipeline.
Recommendations	The recommendation engine is future-facing.	Use analytics outputs for personalized recipe ranking.
UI Polish	Some forms can be refined.	Improve responsive layout, validation messages, and image/text controls.

14. Conclusion:

Savr meets the main goals of the project by delivering a working recipe-sharing social media application with authentication, recipe creation, recipe feed and details, saved recipes, follow functionality, comments, ratings, delete recipe behavior, sorting/statistics, image search, analytics, and lakehouse support. The final deliverable demonstrates software engineering practices through modular architecture, database design, traceability, testing, documentation, Docker setup, and design artifacts. The project is suitable for final evaluation because the implemented functionality is tied directly to the RTM and supported by tests or demo evidence.